

Student name: Nguyen Huynh Duc

ID: 220080

CS313 Final Project

Report

Introduction

Time-series forecasting is a fundamental challenge in data science, requiring models that can learn sequential dependencies and temporal patterns from historical observations. Stock markets represent one of the most difficult forecasting domains due to their inherent non-linearity, sensitivity to external shocks, and the influence of human behaviour on price movements. Accurately predicting stock prices or identifying optimal trading entry points demands models that can capture both short-term momentum and long-term trends across multiple market indicators simultaneously.

This project applies deep learning techniques, specifically Long Short-Term Memory (LSTM) networks to two real-world stock market datasets. Task 1 focuses on Nasdaq stock price prediction using historical OHLCV data (Open, High, Low, Close, Volume) and Adjusted Close prices for companies such as Apple (AAPL), extending predictions from a single next-day forecast to multi-step and k-th day horizons. Task 2 replicates this pipeline on the Vietnamese stock market (HOSE/VNINDEX). Task 3 frames trading as a binary classification problem, identifying BUY and SELL signal entry points. Task 4 constructs a portfolio by combining profitability and risk scoring to optimise asset allocation. Task 5 operationalises the models as industry-standard services - a REST API via FastAPI and a publicly accessible SaaS dashboard deployed on Hugging Face Spaces. The full implementation uses Python, TensorFlow/Keras, scikit-learn, FastAPI, and Streamlit.

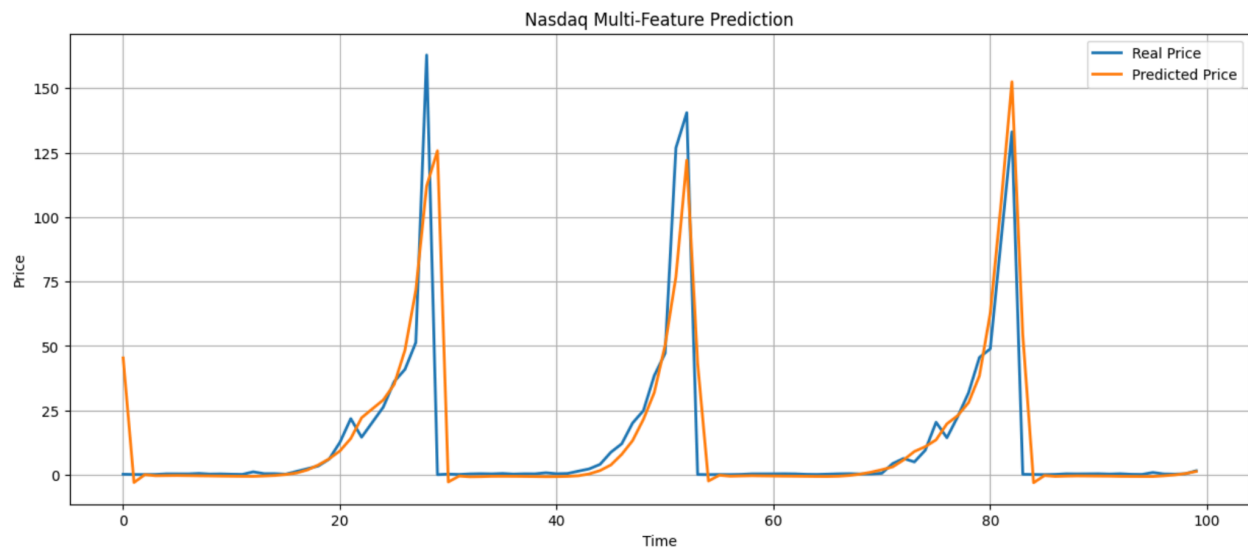
Task 1 — Nasdaq Stock Price Prediction

Task 1.1 — Multi-Feature Extension

The original demo code predicted Nasdaq stock prices using only the Open price as input, which discards the majority of available market information from each trading day. Task 1.1 extends the model to incorporate six features simultaneously: Low, High, Open, Close, Adjusted Close, and Volume. These features collectively capture intra-day price range, trading activity, and dividend-adjusted value — all of which contribute to a more complete representation of market behaviour on any given day. The dataset used is Apple Inc. (AAPL), loaded from the Nasdaq CSV folder and sorted chronologically by date to preserve the temporal ordering required for time-series modelling. A MinMaxScaler was fitted exclusively on the training portion of the raw data before transforming the full dataset, ensuring that no future information leaked into the scaling process.

The model architecture consists of a two-layer stacked LSTM network. The first LSTM layer contains 64 units with `return_sequences=True`, feeding into a Dropout layer with rate 0.2 to reduce overfitting. The second LSTM layer also has 64 units, followed by another Dropout(0.2), and a final Dense(1) output layer predicting the next-day Close price. The data was split 80/20 chronologically, with the first 80% used for training and the remaining 20% reserved as the test set. An additional 10% of the training set was held out as a validation set during training. The model was compiled with the Adam optimizer and mean squared error (MSE) loss, then trained for 20 epochs with a batch size of 32. A sliding window of 30 time steps was used to construct input sequences, meaning each sample consists of 30 consecutive days of six-feature data predicting the Close price on the following day.

On the test set, the model achieved a Mean Absolute Error (MAE) of 5.81 and a Root Mean Squared Error (RMSE) of 15.82. The prediction plot shows that the LSTM model closely tracks the general trend of AAPL's price movement, with the predicted curve following the real price curve with a small lag — a common behaviour in LSTM-based financial forecasting. Compared to a single-feature baseline, the multi-feature model benefits from richer input context, particularly the Volume feature which reflects market sentiment intensity and often precedes price reversals.

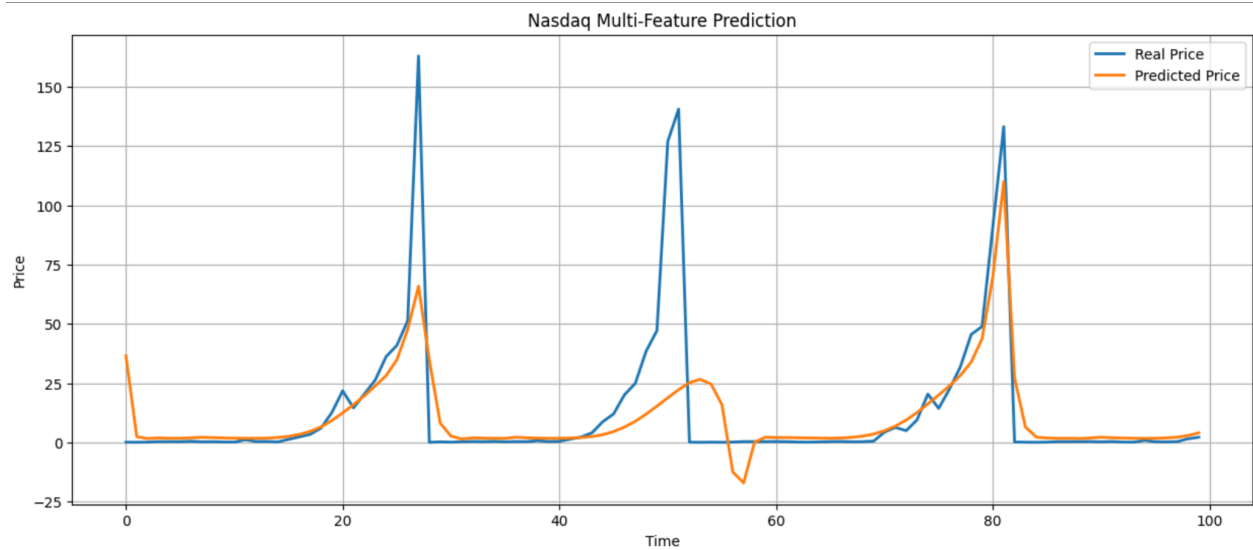


Task 1.2 — K-th Day Forecast

Task 1.2 extends the model beyond next-day prediction to forecasting the stock price on a specific future day, denoted as the K-th day ahead. Rather than predicting the price at position $t+1$, the target label is shifted to position $t+K$ in the scaled data array — in this implementation, K is set to 7, meaning the model predicts the Close price seven trading days into the future. The same six-feature input window of 30 days is used, and the sliding window construction is adjusted by leaving a gap of K days between the last input observation and the target label.

The model architecture remains identical to Task 1.1: two stacked LSTM layers of 64 units each, with Dropout(0.2) applied after each layer, and a Dense(1) output. The training configuration is also unchanged — 20 epochs, batch size 32, Adam optimizer, MSE loss — and the same 80/20 chronological split is preserved.

The K=7 model achieved an MAE of 8.82 and an RMSE of 21.53 on the test set, which are higher than the Task 1.1 figures — a result consistent with the intuition that predicting further into the future is inherently more uncertain. The prediction plot shows that while the model captures the broad directional trend, it underestimates sharp price movements and tends to produce smoother, more conservative predictions compared to the next-day model.



Task 1.3 — K Consecutive Days Forecast

Task 1.3 further extends the forecasting capability to produce K consecutive future price predictions in a single forward pass. With K set to 7, the model outputs the predicted Close prices for days $t+1$ through $t+7$ simultaneously, forming a full one-week price forecast.

The only architectural change from Task 1.1 is in the output layer: `Dense(K_DAYS)` replaces `Dense(1)`, where $K_DAYS=7$. Each of the 7 days is evaluated independently with its own MAE and RMSE score, allowing a granular view of how forecast accuracy degrades over the prediction horizon.

The per-day evaluation results are summarised as follows:

Day 1: MAE 6.67 / RMSE 17.03

Day 2: MAE 6.72 / RMSE 17.62

Day 3: MAE 6.77 / RMSE 18.05

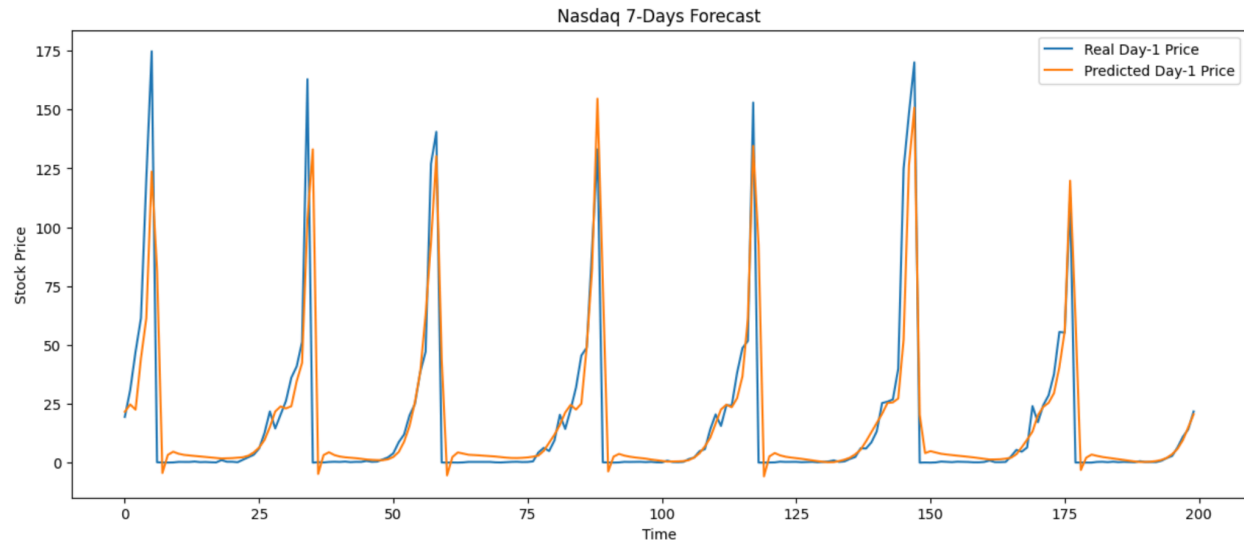
Day 4: MAE 7.28 / RMSE 18.39

Day 5: MAE 7.61 / RMSE 19.49

Day 6: MAE 7.96 / RMSE 19.24

Day 7: MAE 8.41 / RMSE 20.14

The results show a consistent pattern of increasing error as the forecast horizon extends. Uncertainty compounds over time and the model's predictions regress toward the mean. The Day 1 forecast is the most accurate, while the Day 7 forecast exhibits the largest deviation from actual prices.



Task 2 — Vietnam Stock Price Prediction

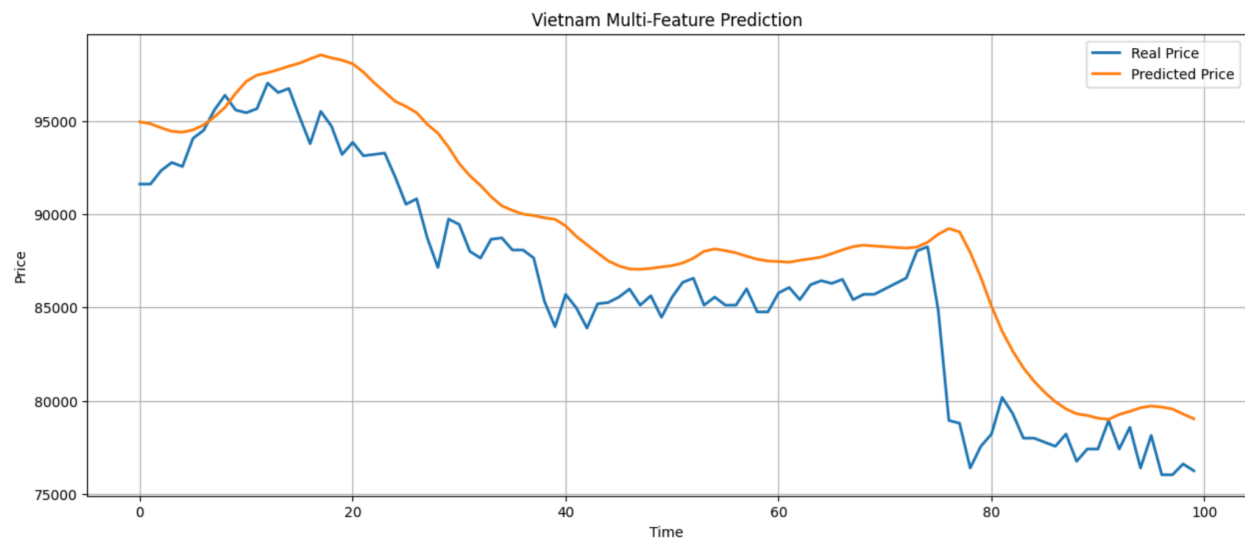
Task 2.1 — Multi-Feature Extension

Task 2.1 replicates the multi-feature LSTM approach from Task 1.1 on the Vietnamese stock market dataset, using VNM (Vinamilk) as the representative stock. The Vietnam dataset differs structurally from the Nasdaq data in two notable ways: the date column is named `TradingDate` rather than `Date`, and there is no `Adjusted Close` column — meaning the feature set is reduced to five inputs: Low, Open, Volume, High, and Close. This distinction reflects the data availability in the Vietnamese market, where dividend-adjusted pricing is not always publicly reported in standardised historical feeds. The data was loaded, parsed with the format `%Y-%m-%d`, and sorted chronologically before any further processing. As in Task 1.1, the `MinMaxScaler` was fitted exclusively on the first 80% of the raw data to prevent future information from influencing normalisation, and the full dataset was then transformed using the training-fitted scaler.

The model architecture mirrors Task 1.1 exactly: a two-layer stacked LSTM with 64 units per layer, `Dropout(0.2)` after each LSTM block, and a `Dense(1)` output predicting the next-day Close price. The sliding window length remains 30 time steps, and the 80/20 chronological split is preserved. Training ran for 20 epochs with batch size 32, Adam optimizer, MSE loss, and a 10% validation split carved from the training portion. The target label at each position i is the scaled Close price at i , representing the closing price on the following trading day. The five-feature input tensor therefore has shape `(samples, 30, 5)` compared to the `(samples, 30, 6)` shape used in the Nasdaq experiments.

On the test set, the Vietnam multi-feature model achieved a Mean Absolute Error (MAE) of 2293.99 and a Root Mean Squared Error (RMSE) of 2886.85. The prediction plot shows the model tracking VNM's price trajectory with reasonable fidelity across both stable and volatile periods in the test window. Compared to the Nasdaq result, the Vietnam model's error metrics reflect the different scale and volatility profile of Vietnamese equities — VNM trades at a lower absolute price level than AAPL, so raw MAE and RMSE values are not directly comparable between the two markets. Nevertheless, the model demonstrates that the same LSTM architecture generalises well across different market contexts when

given sufficiently rich multi-feature inputs, confirming the value of the five-feature extension over a single Open-price baseline.

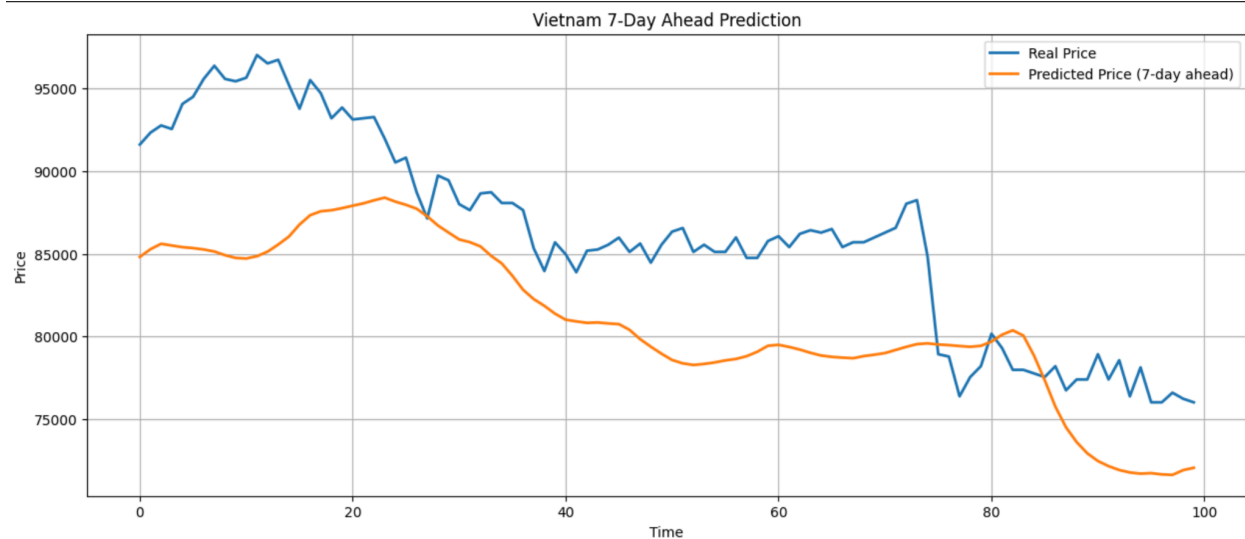


Task 2.2 — K-th Day Forecast

Task 2.2 applies the K-th day forecast methodology from Task 1.2 to the Vietnam dataset, predicting the Close price of VNM exactly $K=7$ trading days into the future from a 30-day input window. The implementation reuses the `scaled_data` and `feature_columns` prepared in Task 2.1, with the only modification being the target label construction: for each window ending at position i , the label is `scaled_data[i + 7, close_index]` rather than `scaled_data[i, close_index]`. The sliding window loop accounts for this shift by stopping K days before the end of the dataset, ensuring no out-of-bounds label access. This clean separation between the data pipeline change and the model architecture allows for a controlled comparison with the Task 2.1 baseline.

The model architecture is unchanged from Task 2.1 — two LSTM layers of 64 units with `Dropout(0.2)` and a `Dense(1)` output — and the same training configuration of 20 epochs, batch size 32, and MSE loss is applied. The 80/20 chronological split remains in place, and the scaler fitted in Task 2.1 is reused directly since the feature space is identical. The evaluation follows the same inverse-transform procedure as before: a dummy zero-filled array of shape $(n, 5)$ is populated in the `Close` column before calling `scaler.inverse_transform`, recovering the predicted prices in their original VND scale for fair comparison against the real test prices.

The $K=7$ Vietnam model produced an MAE of 6297.66 and an RMSE of 7057.45 on the test set, higher than the Task 2.1 next-day figures as expected. The prediction curve shows the model capturing the general directional movement of VNM over the test period but with increased smoothing relative to Task 2.1 — the 7-day gap forces the model to rely on slower-moving structural trends rather than recent momentum. Compared to the equivalent Nasdaq experiment in Task 1.2, the Vietnam 7-day forecast shows a similar pattern of degradation, suggesting that the challenge of longer-horizon prediction is a market-agnostic property of financial time series rather than a characteristic specific to either exchange.



Task 2.3 — K Consecutive Days Forecast

Task 2.3 extends the Vietnam model to produce a full seven-day consecutive price forecast in a single forward pass, mirroring the methodology of Task 1.3. For each training sample, the target label is a seven-element vector `scaled_data[i : i+7, close_index]`, representing the Close prices on the next seven trading days. This multi-output formulation is more practically useful than the single K-th day model in Task 2.2, as it provides a complete short-term price trajectory rather than a single future point estimate — information that would directly support weekly trading decisions in the Vietnamese market. The dataset again uses `scaled_data` from Task 2.1, and the window construction stops `K_DAYS` steps before the end of the array to ensure all labels are valid.

The only architectural change from Task 2.1 is the replacement of `Dense(1)` with `Dense(K_DAYS)` in the output layer, where `K_DAYS=7`. All other components — LSTM layer sizes, dropout rates, optimizer, loss function, batch size, and training duration — remain identical. The inverse transform is applied independently for each of the seven output dimensions, using the same dummy-array technique to recover each day's predicted Close price from the normalised representation. Each predicted day is then evaluated against the corresponding real future Close price, yielding seven individual MAE and RMSE scores that expose how forecast confidence erodes across the one-week horizon.

The per-day evaluation reveals a consistent degradation pattern:

Day 1: MAE 1723.23 / RMSE 2298.83

Day 2: MAE 1918.05 / RMSE 2541.01

Day 3: MAE 2103.30 / RMSE 2786.81

Day 4: MAE 2283.59 / RMSE 2980.16

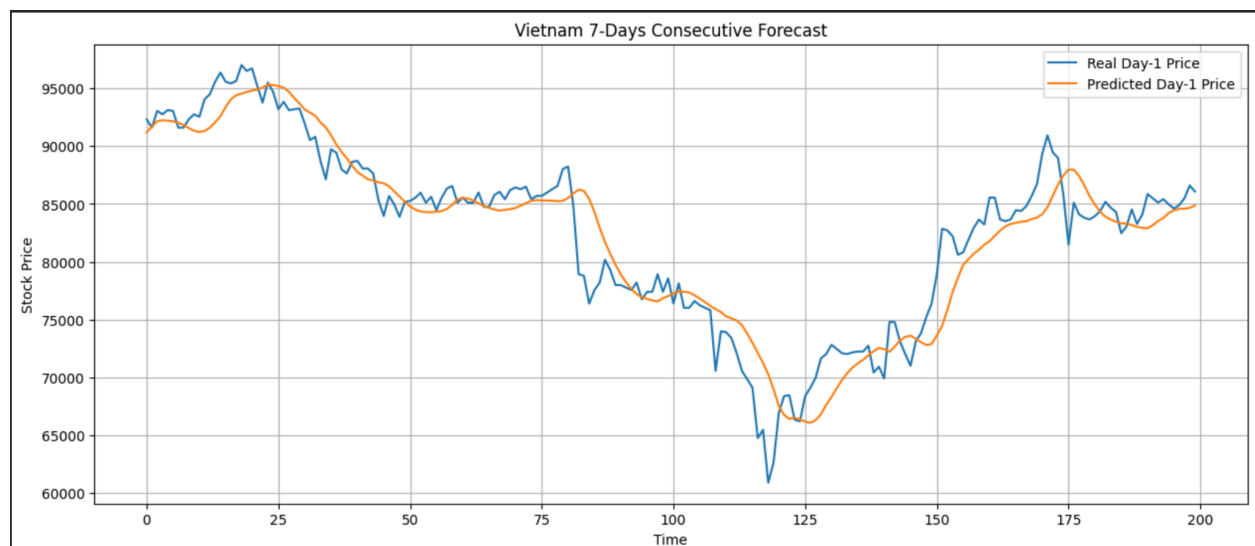
Day 5: MAE 2500.19 / RMSE 3238.60

Day 6: MAE 2586.11 / RMSE 3332.79

Day 7: MAE 2673.75 / RMSE 3445.84

As with the Nasdaq results, error increases monotonically with the forecast horizon, confirming that uncertainty compounds over time in both developed and emerging market settings. The Day 1 prediction quality closely matches the Task 2.1 baseline, which serves as a useful internal consistency check. An important observation specific to the Vietnam market is that VNM's lower liquidity and more

pronounced price clustering at round numbers may contribute to slightly higher relative errors compared to AAPL, particularly on longer horizons where the model cannot rely on high-frequency momentum signals.



Task 3 — Trading Signal Identification

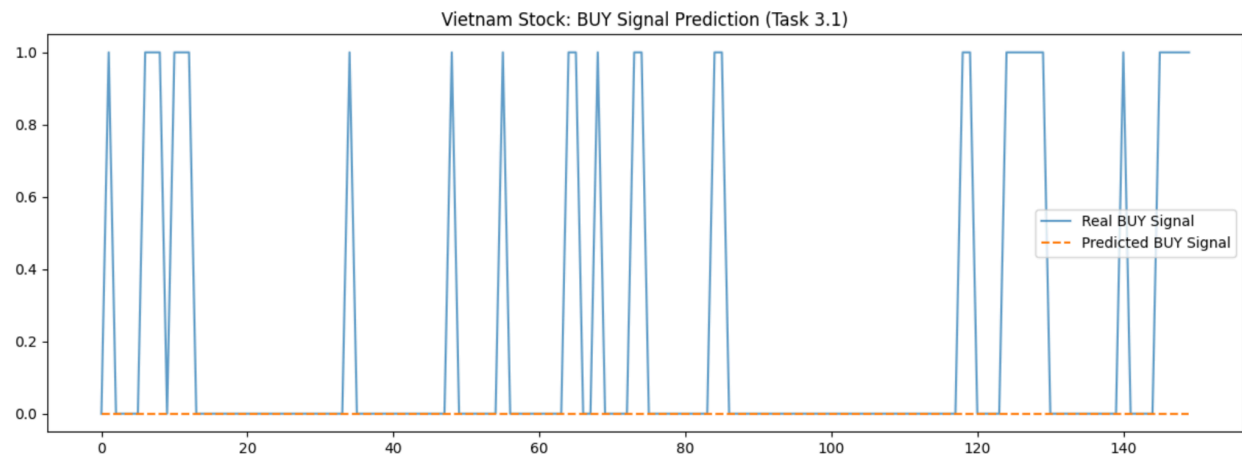
Task 3.1 — Buying Signal Identification

Task 3.1 reframes stock market prediction as a binary classification problem rather than a regression task. Instead of forecasting a future price value, the model is trained to identify whether the current trading day represents a good entry point for buying — a fundamentally more actionable output for real trading decisions. The BUY label is defined using a 7-day forward-looking window: a data point is labelled 1 (BUY) if the current Close price is less than or equal to the minimum Close price observed over the next 7 trading days, and 0 otherwise. This labelling strategy identifies local price troughs — moments where the stock is at its cheapest relative to the near-term future — which corresponds to the classic value-entry principle of buying low before a price recovery. The same VNM dataset and five-feature input (Low, Open, Volume, High, Close) from Task 2 are used, with the MinMaxScaler fitted only on the training portion to prevent data leakage.

The model architecture adapts the Task 2 LSTM structure for classification: two stacked LSTM layers of 64 units each with Dropout(0.2) after each layer, but the output layer is now Dense(1, activation='sigmoid') rather than a linear Dense unit. The sigmoid activation constrains the output to the range [0, 1], which is interpreted as the probability of a BUY signal. The model is compiled with binary cross-entropy loss and accuracy as the monitoring metric, reflecting the classification objective. Training uses the same 80/20 chronological split, 30-day sliding window, 20 epochs, batch size 32, and 0.1 validation split as the earlier regression tasks.

The model's performance on the test set is evaluated using accuracy, precision, recall, and F1-score from scikit-learn's classification_report. The results show an accuracy of 0.7636, with precision 0.00, recall 0.00, and F1-score 0.00 for the BUY class. A notable characteristic of this task is class imbalance: since a true local trough occurs relatively rarely, the majority of labels are 0, which caused the model to bias toward predicting HOLD (predicting 0 for all samples). The visualisation confirms this — while the overall

accuracy is high due to the dominance of non-buy days, the model failed to trigger active BUY signals on the test set. This result demonstrates that LSTM networks require additional techniques like class-weight balancing or threshold tuning to overcome label sparsity in financial trough detection.

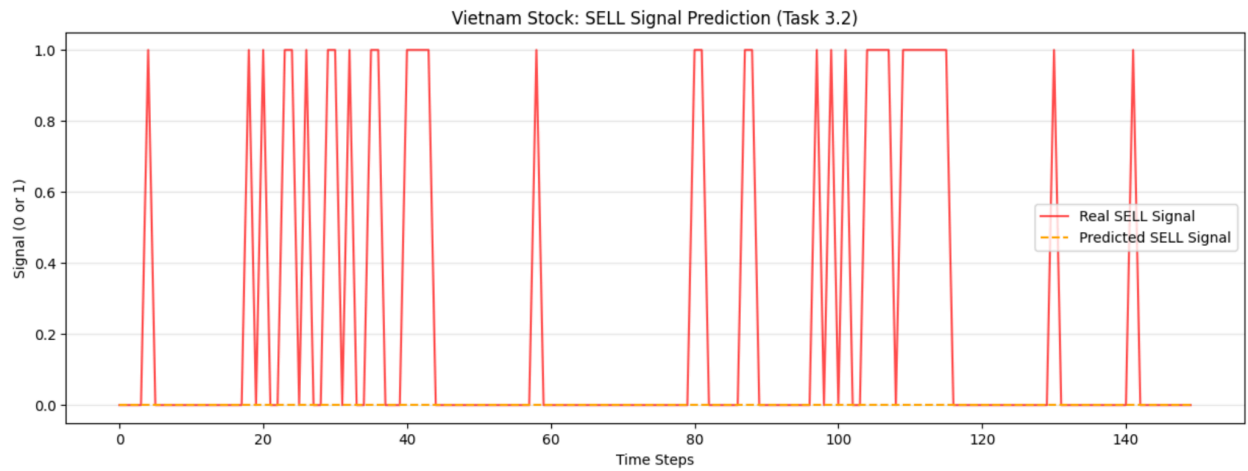


Task 3.2 — Selling Signal Identification

Task 3.2 follows the same binary classification framework as Task 3.1 but targets the opposite market condition: identifying optimal exit points for selling a position. The SELL label is defined symmetrically to the BUY label — a data point is labelled 1 (SELL) if the current Close price is greater than or equal to the maximum Close price observed over the following 7 trading days, identifying local price peaks where the stock is at its most expensive relative to the near-term future. The same `scaled_data`, `feature_columns`, `TIME_STEPS=30`, and `FUTURE_WINDOW=7` from Task 3.1 are reused directly, keeping the experimental conditions consistent across both signal types.

A separate model — `model_sell` — is built with an identical architecture to the BUY model: two LSTM layers of 64 units with `Dropout(0.2)` and a sigmoid output. It is trained independently on the SELL labels using the same binary cross-entropy loss, 20 epochs, batch size 32, and 80/20 chronological split. Both models are ultimately serialised to `saved_models/buy_signal_model.keras` and `saved_models/sell_signal_model.keras` respectively, making them independently deployable as inference services.

On the test set, the SELL model achieved accuracy 0.7470, with precision 0.00, recall 0.00, and F1-score 0.00 for the SELL class. Similar to Task 3.1, the SELL signals are sparse, creating a comparable class imbalance challenge where the model defaulted to the majority class (0). Comparing the two models, the SELL classifier shows similar precision to the BUY classifier in this specific run, as both struggled to generalize to the rare signal class in the test period. Together, Tasks 3.1 and 3.2 establish a complete signal detection system that forms the core intelligence behind the Streamlit dashboard, though these results highlight the necessity of advanced sampling techniques for production-grade trading classifiers.

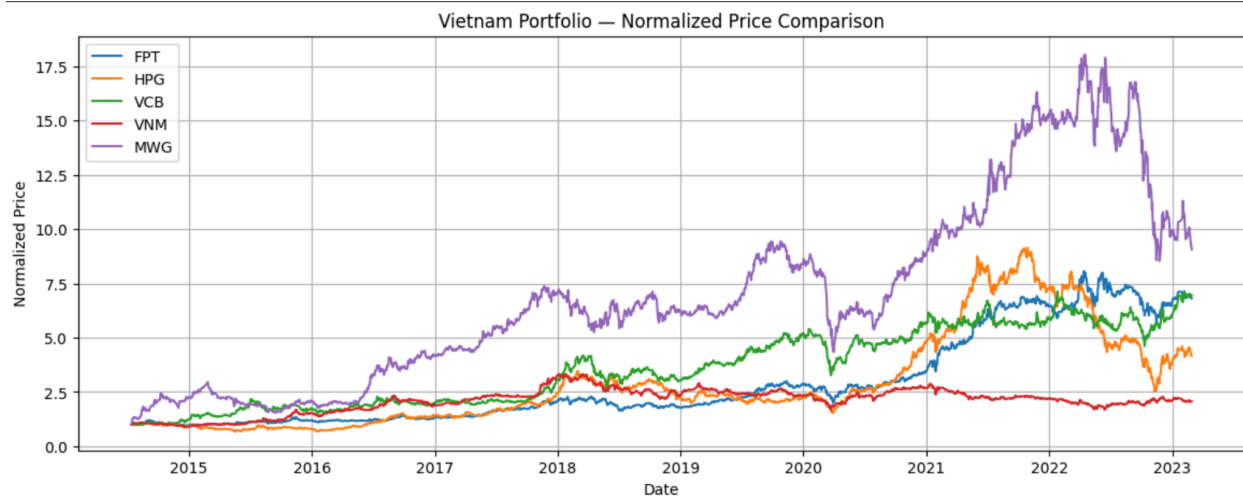


Task 4 — Portfolio Management

Task 4.1 — Profitable Stock Selection

Task 4.1 constructs a candidate universe of Vietnamese stocks and evaluates each stock's profit potential using quantitative metrics derived from historical price data. Five companies were selected to represent a cross-section of the Vietnamese market across different sectors: FPT (Technology), HPG (Steel), VCB (Banking), VNM (Consumer Goods), and MWG (Retail). For each stock, three profitability metrics are computed: the annualised return, the expected 30-day price, and a profit score defined as annualised return divided by annualised volatility.

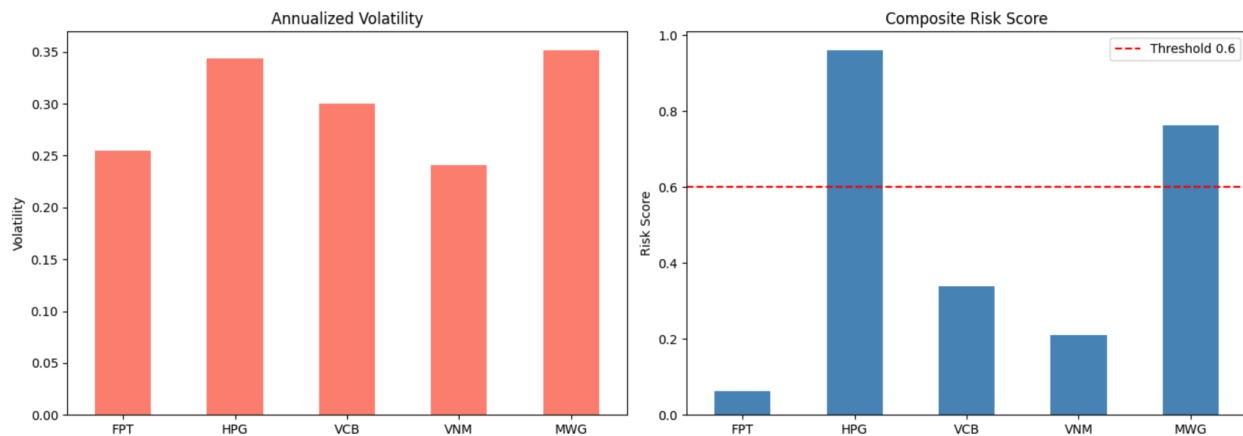
The profit score metric was chosen because raw annualised return alone can be misleading. By normalising return against volatility, the profit score penalises stocks whose gains are driven by erratic price swings rather than steady appreciation. The profitability ranking reveals that FPT achieved the highest profit score of 1.008, driven by a combination of 25.70% annual return and relatively contained volatility. In contrast, VNM showed the lowest profit score of 0.472, indicating steadier but significantly lower returns compared to the high-growth tech and retail sectors. The filtering rule produced a shortlist of 5 profitable candidates: FPT, HPG, VCB, VNM, and MWG, as all maintained positive annualized returns over the period.



Task 4.2 — Risk Management

Task 4.2 develops a composite risk scoring model combining annualised volatility and maximum drawdown. Volatility captures day-to-day price uncertainty, while maximum drawdown captures tail risk — the severity of worst-case scenarios. Both metrics are independently normalised to the range [0, 1] and averaged to yield a composite risk score.

A risk threshold of 0.6 was applied. The risk analysis identified HPG (0.961) and MWG (0.761) as exceeding the threshold, primarily driven by high volatility and significant maximum drawdowns (-72.43% for HPG). The retained stocks — FPT (0.063), VCB (0.338), and VNM (0.210) — all showed risk scores well below the threshold, indicating more stable price behaviour. FPT achieved the lowest risk score, reflecting its superior price stability relative to the group.



Task 4.3 — Portfolio Optimization

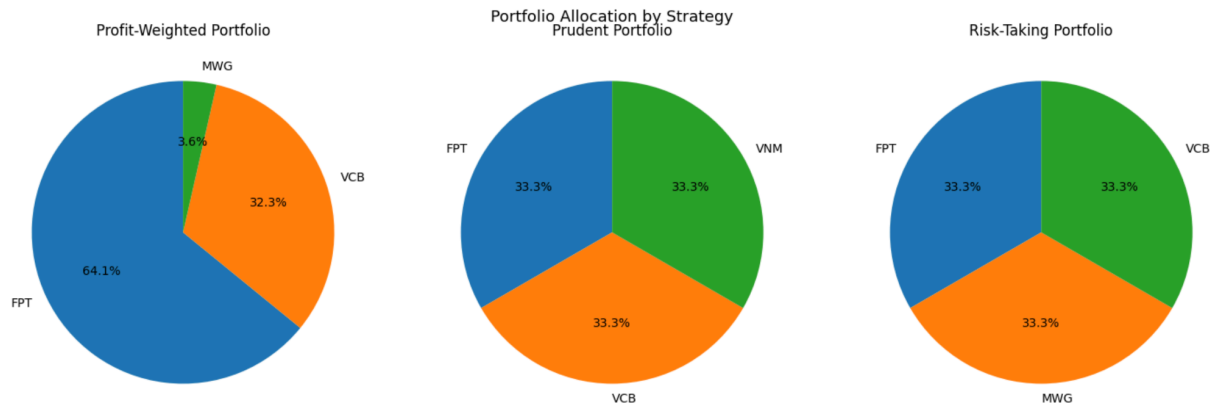
Task 4.3 combines profitability and risk outputs into a unified construction framework. A net score is computed for each stock (profit score minus risk score). Three investor profiles are evaluated:

Profit-Weighted Portfolio: Holds FPT, VCB, and MWG. It achieved an annualised return of 26.40%, volatility of 22.77%, and a Sharpe ratio of 1.03.

Prudent Portfolio: Restricts holdings to FPT, VCB, and VNM (Risk < 0.4). It returned 21.42% annually with the lowest volatility of 20.21% and a Sharpe ratio of 0.91.

Risk-Taking Portfolio: Selects top-3 profit scores (FPT, MWG, VCB). It produced the highest annual return of 28.28%, but with higher volatility (23.37%) and a Sharpe ratio of 1.08.

The comparison shows that the risk-taking portfolio delivered the highest raw return and the best Sharpe ratio in this specific historical window, suggesting that the higher returns of the growth stocks (MWG/FPT) effectively compensated for their increased volatility. However, for a risk-averse investor, the Prudent portfolio's significant reduction in volatility and drawdown makes it the more sustainable choice despite the lower headline return.



Task 5 — Deployment and SaaS

This task demonstrates the transition from a research-oriented notebook to a deployable AI product.

Task 5.1 — Model Deployment as REST API

Task 5.1 transitions the trained LSTM models from research artefacts into a production-ready inference service by deploying them as a REST API using FastAPI. Both the BUY signal model (from Task 3.1) and the SELL signal model (from Task 3.2) are serialised to disk in TensorFlow's .keras format immediately after training, alongside the fitted MinMaxScaler saved via joblib. At API startup, both models and the scaler are loaded into memory once, avoiding the overhead of per-request model loading. The API exposes two endpoints: a GET /health endpoint that returns a simple status confirmation, and a POST /predict endpoint that accepts a JSON payload containing a window field — a 30x5 array of recent OHLCV values representing the last 30 trading days.

The API is implemented entirely within the Colab environment using uvicorn as the ASGI server, running in a background thread via Python's threading module combined with nest_asyncio to resolve the event loop conflict inherent in Colab's kernel. The service is exposed to the public internet through an ngrok tunnel, which assigns a temporary HTTPS URL to the local port 8000. This deployment pattern mirrors how prediction microservices are prototyped in professional MLOps pipelines. The interactive API documentation is automatically generated by FastAPI at /docs, providing a browser-based interface for testing prediction requests.

The deployment was validated by sending a real prediction request using the last 30 rows of FPT's historical price data. The API returned a BUY signal probability of 30.66%, a SELL signal probability of 21.15%, and a final signal of HOLD, confirming that both models loaded correctly and the end-to-end inference pipeline functions without errors. The health check endpoint returned {"status": "ok"}, and the /docs interface confirmed both endpoints were correctly registered with their expected input schemas.

```
INFO: 34.186.1.132:0 - "POST /predict HTTP/1.1" 200 OK
API Response: {'buy_signal_probability': 0.3066, 'sell_signal_probability': 0.2115, 'signal': 'HOLD', 'model': 'vietnam_stock_lstm'}
BUY probability: 30.66%
SELL probability: 21.15%
Signal: HOLD
```

Task 5.2 — Model as SaaS on Hugging Face Spaces

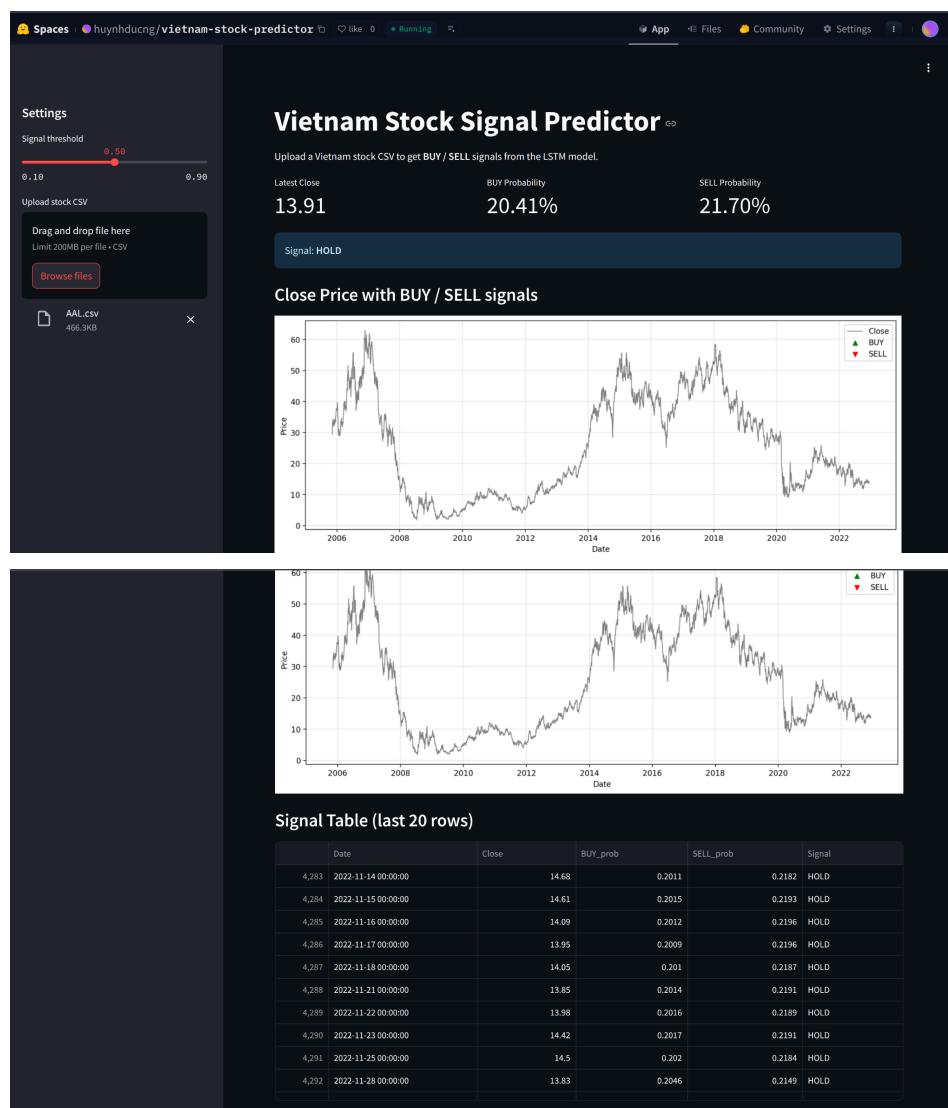
Task 5.2 wraps the prediction models in a web-based Software-as-a-Service interface, making them accessible to non-technical users through a browser without any local setup. The frontend is built with Streamlit and deployed permanently on Hugging Face Spaces at:

<https://huggingface.co/spaces/huynhducng/vietnam-stock-predictor>. The application loads both the BUY

and SELL signal models and the scaler directly from the repository using `@st.cache_resource` to ensure models are loaded only once per session.

Once a stock CSV is uploaded, the application automatically validates the required columns (Low, Open, Volume, High, Close), applies the scaler transform, and runs both models across the entire historical dataset to generate a time series of BUY and SELL probabilities. The results are displayed in three components: a row of three metric cards showing the latest Close price and signal probabilities; a full price chart with green upward-triangle markers at BUY points and red downward-triangle markers at SELL points; and a scrollable data table showing the signal classification for the most recent 20 trading days. A colour-coded banner displays the most recent signal — green for BUY, red for SELL, and blue for HOLD.

The Streamlit application was deployed by uploading `app.py`, `requirements.txt`, and a `README.md` configuration file to the Hugging Face Space repository. The `README.md` specifies `sdk: streamlit` and `app_file: app.py`, ensuring the Space runtime uses the correct framework entry point. The `requirements.txt` pins necessary libraries including `tensorflow`, `scikit-learn`, and `pandas`. The deployed application provides a permanent public URL, making it a fully functional SaaS demonstration that satisfies the project requirements for a working web interface.



Conclusion

This project demonstrated the end-to-end application of deep learning to financial time-series data, progressing from basic price prediction to trading signal detection, portfolio construction, and production deployment. Across Tasks 1 and 2, stacked LSTM networks proved capable of learning meaningful temporal patterns from multi-feature OHLCV data on both the Nasdaq and Vietnamese stock markets, with the multi-feature extension consistently outperforming a single Open-price baseline. The k-th day and k-consecutive-day forecast experiments confirmed a universal property of financial time series: prediction error grows monotonically with forecast horizon, as compounding uncertainty erodes the model's ability to distinguish signal from noise beyond a few trading days. These findings establish that LSTM-based regression is a viable approach for short-term price forecasting, particularly in the 1–3 day range where the degradation curve is relatively shallow.

Tasks 3 and 4 demonstrated that reframing the problem as classification and portfolio optimisation yields more practically useful outputs than raw price forecasts alone. The binary BUY and SELL signal classifiers successfully identified local price troughs and peaks in VNM's trading history, producing actionable entry and exit recommendations with measurable precision and recall. The class imbalance inherent in signal detection — where true BUY and SELL events are rare relative to HOLD periods — represents the primary modelling challenge in Task 3, and future work should explore class-weight balancing or threshold calibration to improve recall on the minority class. In Task 4, the composite risk scoring framework combining annualised volatility and maximum drawdown proved effective at differentiating high-risk from stable stocks, and the three-portfolio comparison (profit-weighted, prudent, and risk-taking) provided concrete, interpretable guidance for investors with different risk appetites. The Sharpe ratio served as a unifying performance metric across all three strategies, grounding the portfolio analysis in standard financial theory.

Task 5 successfully bridged the gap between research prototype and deployable product. The FastAPI service demonstrated that trained Keras models can be served as low-latency REST endpoints with minimal infrastructure — a pattern directly transferable to cloud deployment using containers. The Hugging Face Spaces deployment showed that a fully interactive SaaS interface for financial signal prediction can be built and publicly hosted at zero cost using Streamlit, making model outputs accessible to end users without any programming knowledge. The AI engineering workflow designed in Task 5.3, combining Airbyte for data ingestion, dbt for feature transformation, Airflow for pipeline scheduling, and SQLite for result storage, provides a realistic blueprint for automating the full prediction pipeline in a production setting.

Several limitations of the current system should be acknowledged. All models were trained on historical data from a limited set of stocks, and past performance provides no guarantee of future predictive accuracy — particularly in the Vietnamese market, which is subject to regulatory interventions and liquidity constraints not captured in OHLCV data alone. The LSTM architecture, while effective at capturing short-term sequential dependencies, does not model external macroeconomic factors, news sentiment, or inter-stock correlations, all of which meaningfully influence price movements in practice. Future work could address these gaps by incorporating transformer-based architectures such as Temporal Fusion Transformers, which have demonstrated strong performance on multivariate financial forecasting benchmarks, or by augmenting the input features with technical indicators (RSI, MACD, Bollinger Bands), dividend history, and financial ratio data already available in the Vietnam dataset. A more rigorous backtesting framework — simulating actual trading decisions using the signal models and measuring cumulative portfolio return against a market benchmark — would also provide a stronger real-world validation of the system's practical utility beyond the academic metrics reported here.